TASK 4: Control Implicit Conversions

Name: Nagineni Sunil Kumar
Register Number: 192472330

1. Aim

To understand implicit type conversions, integer division, signed and unsigned comparisons, narrowing conversions, and explicit casting in C++.

2. C++ Program

```
#include <iostream>

using namespace std;

int main() {

   int a = 7, b = 2;

   double c = a / b;

   double d = static_cast<double>(a) / b;

   cout << c << ' ' << d << '\n';

   char ch = 'A' + 2;

   cout << ch << ' ' << ch + 1 << '\n';

   bool flag = 5;

   cout << flag << ' ' << boolalpha << flag << '\n';

   unsigned int u = 1;

   int s = -1;

   cout << (s < u ? "s < u" : "s >= u") << '\n';}
```

3. Predicted Output

3 3.5

C 68

1 true

s >= u

Obtained output:



4. Explanation

1. a / b uses integer division, so 7 / 2 produces 3. That result is then converted to double. The explicit cast makes the second division produce 3.5.
2. 'A' + 2 produces the character C. In ch + 1, the character is promoted to an integer, producing 68 on systems using the usual ASCII-compatible character encoding.
3. The integer 5 converts to true. By default, true prints as 1; after boolalpha, it prints as true.
4. In the comparison s < u, the negative signed value is converted to unsigned for the comparison. The result is false, so the program prints s >= u. Compiling with warnings may flag the signed/unsigned comparison.

5. Narrowing Conversions

int x = 3.7; // Converts to 3; fractional part is lost.

// int y{3.7}; // Compile-time error: narrowing conversion

Brace initialisation can catch certain narrowing conversions at compile time.

6. Alternative Concept and Complexity

Using explicit static_cast expressions makes conversions visible. Brace initialisation helps detect some unsafe conversions. These changes generally do not affect the algorithm's time or space complexity, but they can catch mistakes earlier.

7. Conclusion

Implicit conversions can cause unexpected results. Explicit casting, suitable data types, compiler warnings, and brace initialisation improve the safety and clarity of C++ programs.